CS 576: Group Assignment 2
UDP Sockets
4/10/2026
Shreya Sridharan, Aj Gabriel, Calvin Rull, Jorge Nunez,
Kenneth Swan, Kurt Lara-Rosales, Miguel Melo Ochoa
git repo link: https://github.com/CS576Group3/UDPSockets.git

Introduction:
In this assignment, we implemented a UDP-based client-server application. The client sends a text message to the server using a UDP socket. The server receives the message, appends a message to the original text, and sends the combined response back to the client. The client then receives the response and displays it on the screen.

Design Overview:

The system contains two programs: the server program and the client program.

Server program:

- Waits for incoming UDP datagrams
- Receives message from client
- Appends joke message to the original message
- Sends the modified message back to the client

Client program

- Creates a UDP socket
- Accepts user input
- Sends the message to the server
- Receives the server's response
- Displays the returned message

Both programs use IPv4 and UDP.

UDP Communication Model:

This is the order in which communication occurs:

- The server creates a UDP socket
- Server binds to (host, port)
- Client creates a UDP socket
- Client sends a datagram to the server using sendto()
- Server receives the datagram using recvfrom()
- Server processes the message and sends a response using sendto()
- Client receives the response using recvfrom()
- Sockets are closed when communication is complete

UDP is a message-based protocol, so it preserves message boundaries. Each call to sendto() sends one complete datagram. Each call to recvfrom() receives one complete datagram up to the buffer size.

Server Implementation:

- Socket Setup

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

  - AF_INET: IPv4
  - SOCK_DGRAM: UDP

- Binding and Listening

```
server_address = ('localhost', 8888)
server_socket.bind(server_address)
```

  - bind() : associates socket with specific port

- Receiving Data

```
message, client_address = server_socket.recvfrom(1024)
```

  - recvfrom() receives a UDP datagram
  - Returns both the message and the sender's address

- Processing the message

```
funnyBit = " - I'm going to break your legs now!"
response = message.decode()
response += funnyBit
```

  - Server takes original message and concatenates with funny response

Client Implementation:

- Creating Socket

```
client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

  - Client initiates a UDP socket for sending and receiving datagrams

- Sending Message

```
client_socket.sendto(message.encode(), server_address)
```

  - The client sends the user's message directly to the server without establishing a connection

- Receiving response:

```
response, _ = client_socket.recvfrom(1024)
```

  - The client waits for the server's reply and decodes the received bytes into string.

Testing:

- Test 1: "Hello"
  -
```
Received message from client: Hello
Test 1: PASS
  Sent:     Hello
  Received: Hello - I'm going to break your legs now!
```

- Test 2: "Hi"
  -
```
Received message from client: Hi
Test 2: PASS
  Sent:     Hi
  Received: Hi - I'm going to break your legs now!
```

Limitations:

- UDP does not guarantee message delivery
- UDP does not guarantee messages come in in order
- No retransmission or acknowledgement system was implemented

Source Code:

Client:

```
import socket

# Form a UDP connection with server
def main():
    # Create a UDP socket
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    # Server address and port
    server_address = ('localhost', 8888)

    try:
        # Send a message to the server
        message = input("Enter a message to send to the server: ")
        print(f"Sending message to server: {message}")
        client_socket.sendto(message.encode(), server_address)

        # Receive a response from the server
        response, _ = client_socket.recvfrom(1024)
        print(f"Received response from server: {response.decode()}")

    except Exception as e:
        print(f"Client error: {e}")

    finally:
        client_socket.close()

if __name__ == "__main__":
    main()
```

Server:

```
import socket

# Form a UDP connection with client
def main():
    # Create a UDP socket
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    # Bind the socket to an address and port
    server_address = ('localhost', 8888)
    server_socket.bind(server_address)
    print("Server is listening on port 8888...")

    try:
        while True:
            # Wait for a message from the client
            message, client_address = server_socket.recvfrom(1024)
            print(f"Received message from client: {message.decode()}")

            # Funny response to concatenate with the original message
            funnyBit = " - I'm going to break your legs now!"

            # Send full response back to the client
            response = message.decode()
            response += funnyBit
            server_socket.sendto(response.encode(), client_address)

    except Exception as e:
        print(f"Server error: {e}")

    except KeyboardInterrupt:
        print("\nServer stopped by user.")

    finally:
        server_socket.close()

if __name__ == "__main__":
    main()
```